
SecureVPN

Post-Quantum WireGuard VPN

Network Security - Project Report

Submitted By

Name	Reg. No.
Muhammad Adeel Haider	241541
Wazeema Eeman	241493
Ainish Iqbal	241509
Umar Farooq	241575

Submitted To

Ms. Areeba Fatima
Instructor, Network Security Lab

Department

National Cyber Security Academy
(NCSA)
Air University, Islamabad

May 22, 2026

Contents

1	Executive Summary	3
2	Introduction and Motivation	3
2.1	Background	3
2.2	Problem Statement	4
2.3	Objectives	4
3	System Architecture	4
3.1	High-Level Architecture	4
3.2	Server-Side Components	5
3.2.1	WireGuard Interface	5
3.2.2	Flask REST API	5
3.2.3	Gunicorn WSGI Server	5
3.2.4	Nginx Reverse Proxy	5
3.2.5	Redis	6
3.3	Protocol Sequence Diagram	6
3.4	Client-Side Components	7
3.4.1	Graphical User Interface	7
3.4.2	CLI Interface	7
3.4.3	Crypto Engine	8
3.4.4	Trust-On-First-Use (TOFU) Server Key Pinning	8
3.4.5	AllowedIPs Routing-Loop Fix	8
3.4.6	Multi-Method Handshake Detection	8
3.4.7	OS-Level Kill Switch	9
3.4.8	Bandwidth Monitoring	9
4	Cryptographic Design	9
4.1	Hybrid Post-Quantum Key Exchange	9
4.2	Kyber-512 KEM Implementation	10
4.2.1	Module Learning With Errors (MLWE)	10
4.2.2	Fujisaki-Okamoto Transform	10
4.2.3	PSK Derivation	10
4.3	API Request Authentication	11
4.4	Transport Security	11
5	Security Architecture	11
5.1	Defence-in-Depth Overview	11

5.2	STRIDE Threat Model Analysis	12
5.3	Principle of Least Privilege	13
5.3.1	Dedicated Service Account	13
5.3.2	Sudoers Wrapper Scripts	13
5.4	Rate Limiting	14
5.5	Anomaly Detection and Monitoring	14
5.6	Peer Lifecycle Management	14
5.7	Firewall Configuration	14
6	Deployment and Configuration	15
6.1	Infrastructure	15
6.2	Network Security Group Rules	15
6.3	Automated Installation	16
6.4	Systemd Service Configuration	16
6.5	Client Setup	17
7	Testing and Verification	17
7.1	Server Health Verification	17
7.2	End-to-End Connection Test	18
7.3	Security Control Verification	19
7.4	Cryptographic Performance Benchmarks	19
8	Conclusion and Future Work	20
8.1	Summary	20
8.2	Limitations	20
8.3	Future Work	21

1. Executive Summary

SecureVPN is a post-quantum-hardened Virtual Private Network built on WireGuard, developed as a Network Security course project at the National Cyber Security Academy (NCSA), Air University. The system implements a hybrid classical/post-quantum cryptographic handshake combining WireGuard's X25519 Elliptic Curve Diffie-Hellman with a Kyber-512 Key Encapsulation Mechanism (KEM), providing forward secrecy against both classical and quantum adversaries. The project delivers a complete, deployable VPN solution comprising:

- A hardened Linux VPN server deployed on Microsoft Azure (East US region, Washington)
- A RESTful Flask API served via Gunicorn behind Nginx TLS termination
- A premium Python/Tkinter Windows GUI client with animated status ring, power-button toggle, real-time bandwidth cards, and DPAPI-encrypted credential storage
- Post-quantum Pre-Shared Key (PSK) derivation via Kyber-512 KEM + HKDF-SHA3-256 — the shared secret never traverses the network
- Defence-in-depth security: Redis-backed rate limiting, HMAC-SHA256 request signing with replay prevention, least-privilege service accounts, validated sudoers wrapper scripts, OS-level kill switch, and Trust-On-First-Use (TOFU) server key pinning

The system was fully deployed, tested, and verified end-to-end: a Windows client successfully established an encrypted WireGuard tunnel to the Azure VPN server with a KEM-derived PSK and confirmed internet connectivity routed through the USA (Washington) exit node.

2. Introduction and Motivation

2.1 Background

Virtual Private Networks are a foundational tool in network security, providing encrypted tunnels between endpoints over untrusted networks. Classical VPN protocols (OpenVPN, WireGuard) rely on Diffie-Hellman key exchange and ECDH, both of which are vulnerable to Shor's algorithm running on a sufficiently capable quantum computer. NIST published FIPS 203 (ML-KEM, based on Kyber) in 2024 as the first post-quantum Key Encapsulation Mechanism standard, signalling the urgency of quantum-resistant network security.

2.2 Problem Statement

Current VPN deployments face a *harvest now, decrypt later* threat: adversaries can capture encrypted VPN traffic today and decrypt it once quantum computers become available. For long-lived sensitive data, this represents an immediate risk. Furthermore, most consumer and enterprise VPN solutions lack:

- API-level authentication with request signing
- Automated peer lifecycle management
- Transparent post-quantum cryptographic hardening

2.3 Objectives

This project aims to:

1. Design and implement a WireGuard VPN with post-quantum KEM-derived PSK
2. Build a secure, authenticated REST API for peer management
3. Deploy the system on a production cloud VPS (Azure)
4. Implement defence-in-depth security across all layers
5. Provide a cross-platform Python client for Windows

3. System Architecture

3.1 High-Level Architecture

The system follows a client-server architecture with four distinct layers:

Layer	Components
Client	Python CLI/GUI, crypto engine, WireGuard tunnel manager
Transport	WireGuard (UDP 51820) + HTTPS API (TCP 443)
Server API	Nginx (TLS) → UNIX socket → Gunicorn → Flask
System	WireGuard kernel module, Redis, systemd, UFW firewall

3.2 Server-Side Components

3.2.1 WireGuard Interface

WireGuard runs as a kernel-space VPN interface (`wg0`) on the server, listening on UDP port 51820. The server is assigned the VPN subnet gateway address `10.77.0.1/24`. Each client peer is assigned a unique `/32` address in the `10.77.0.0/24` range. IP forwarding is enabled via `sysctl`, and NAT masquerading is configured via `iptables` PostUp/PostDown hooks in the WireGuard configuration, enabling clients to route all internet traffic through the server.

3.2.2 Flask REST API

The management API is implemented in Python using Flask and provides the following endpoints:

Endpoint	Method	Description
<code>/api/v1/health</code>	GET	Unauthenticated health check
<code>/api/v1/status</code>	GET	WireGuard interface status and peer list
<code>/api/v1/add-peer</code>	POST	Register new peer, trigger KEM PSK derivation
<code>/api/v1/revoke-peer</code>	POST	Remove peer from WireGuard and config
<code>/api/v1/audit-peers</code>	GET	Peer lifecycle audit (active, expired, unknown)
<code>/api/v1/anomalies</code>	GET	Handshake timeout and traffic anomaly detection
<code>/api/v1/pq-keygen</code>	POST	Server-side Kyber-512 keypair generation

3.2.3 Gunicorn WSGI Server

Flask is served via Gunicorn with 4 worker processes (gthread class, 2 threads each) bound to a UNIX domain socket at `/run/wg-api/wg-api.sock`. This socket is never exposed directly to the network; Nginx acts as the sole network-facing endpoint.

3.2.4 Nginx Reverse Proxy

Nginx terminates TLS on port 443 and forwards requests to the Gunicorn UNIX socket. The configuration enforces:

- TLS 1.2/1.3 only, with strong cipher suites

- HSTS with 63-day max-age
- Security headers: `X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy`
- Rate limiting: 20 requests/minute per IP via `limit_req_zone`

3.2.5 Redis

Redis provides the backend storage for Flask-Limiter’s distributed rate limiting. It is hardened to bind only to localhost, with dangerous commands (`FLUSHALL`, `CONFIG`, `DEBUG`, `SHUTDOWN`) renamed to empty strings to prevent abuse if the service is somehow reached.

3.3 Protocol Sequence Diagram

The end-to-end interaction between the Client, API Server, and the WireGuard Interface during peer registration and connection establishment is illustrated below. The REST API facilitates the out-of-band post-quantum key exchange, which then parameterises the in-band WireGuard tunnel.

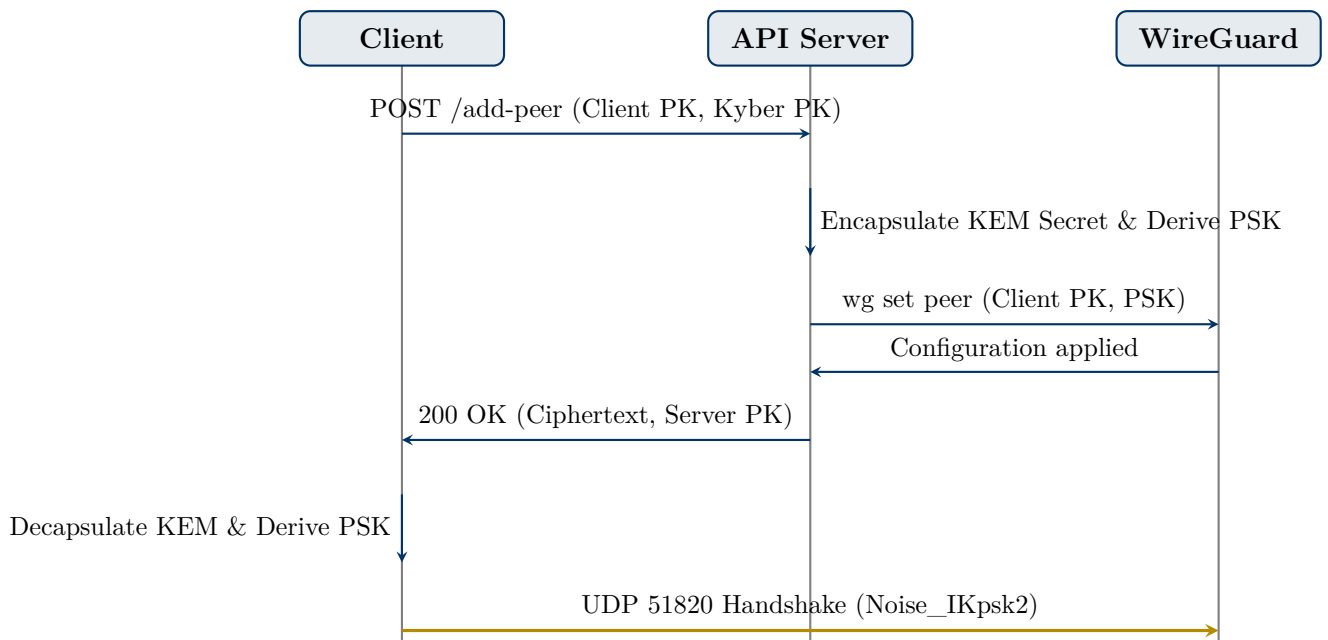


Figure 1: Protocol Sequence Diagram: Hybrid Post-Quantum Registration and Handshake

3.4 Client-Side Components

3.4.1 Graphical User Interface

The Windows client features a premium dark-themed GUI (`tkinter`, 960×640 pixels) inspired by macOS design principles. Key interface elements include:

- **Sidebar navigation:** Connect, Profiles, Logs, Settings pages with animated dot status indicator
- **Animated status ring:** A 260 px Canvas ring shows a spinning dual-arc during *Connecting*, a full glowing ring on *Connected*, and a broken arc on *Disconnected*
- **Power button:** A 110 px Canvas button with layered glow halo; hover and press states change colour; a single click toggles the tunnel
- **Profile dropdown:** Custom popup menu listing all locally stored `.conf` profiles; new profiles are generated via the Profiles page
- **Real-time stat cards:** Server IP, Client VPN IP, last handshake time, and live bandwidth (rx ↓ / tx ↑) updated every five seconds via a background polling thread
- **Pill toggle switches:** Smooth animated Canvas toggles for Kill Switch and Auto-Connect settings
- **Structured log panel:** Colour-coded log viewer (info / success / warning / error) with auto-scroll

All VPN operations (`up`, `down`, `keygen`) execute on daemon threads to keep the UI responsive.

3.4.2 CLI Interface

The Windows client also exposes a command-line interface:

```
1 python securevpn_cli.py keygen <name> --server <url> --api-key
   <key>
2 python securevpn_cli.py up --profile <name>
3 python securevpn_cli.py down --profile <name>
4 python securevpn_cli.py status
5 python securevpn_cli.py list
6 python securevpn_cli.py verify          # DNS/IP leak test
7 python securevpn_cli.py rotate-psk     # Rotate post-quantum PSK
```

Listing 1: SecureVPN CLI usage

3.4.3 Crypto Engine

The client-side crypto engine (`crypto_engine.py`) handles:

- WireGuard keypair generation via `wg genkey` / `wg pubkey` subprocess
- Kyber-512 KEM keypair generation, encapsulation, and decapsulation
- HKDF-SHA3-256 PSK derivation from the KEM shared secret
- Credential storage via Windows DPAPI (`CryptProtectData`) with AES-256-GCM + OS keyring fallback

3.4.4 Trust-On-First-Use (TOFU) Server Key Pinning

On first connection to a server, the client pins the WireGuard server public key to a local `pinned_keys.json` file. On every subsequent up, the engine reads the current key from the profile `.conf` and compares it to the pinned value. A mismatch raises a `SecurityError` which aborts the connection and alerts the user with a warning dialog — providing protection against MITM attacks via modified profile files.

3.4.5 AllowedIPs Routing-Loop Fix

Routing `0.0.0.0/0` through the tunnel includes the VPN server's own public IP, causing the WireGuard UDP packets to be routed into the tunnel they are trying to establish — a routing loop that prevents the handshake from completing (`rx_bytes` remains 0).

The client resolves this with Python's `ipaddress.address_exclude()` to split `0.0.0.0/0` around the server's `/32`, producing the RFC-compliant two-prefix equivalent that excludes exactly the server endpoint IP.

3.4.6 Multi-Method Handshake Detection

After installing the tunnel service, the engine polls for up to 60 seconds using four independent detection methods to confirm the WireGuard handshake actually completed:

1. **wg show dump**: checks for a peer handshake timestamp within the last 180 seconds
2. **Ping 10.77.0.1**: if the VPN gateway responds, the tunnel is provably up
3. **Windows Service check**: `sc query WireGuardTunnel$<name>` for `RUNNING`
4. **Network adapter check**: `ipconfig` scanned for a `10.77.0.x` address

3.4.7 OS-Level Kill Switch

The kill switch prevents traffic leaks if the tunnel drops unexpectedly. It is implemented using Windows Advanced Firewall (`netsh advfirewall`) with seven priority-ordered rules:

Priority	Rule Name	Direction	Action
1000	Allow-Tunnel	Out	Allow all on WireGuard adapter
1001	Allow-VPN	Out	Allow to server endpoint IP
1002	Allow-Loopback	Out	Allow to 127.0.0.1
1003	Allow-DHCP	Out	Allow UDP 68→67
1004	Allow-VPN-Subnet	Out	Allow to 10.77.0.0/24
1005	Allow-DNS-Tunnel	Out	Allow UDP 53 on tunnel adapter
2000	Block	Out	Block ALL remaining outbound

Allow rules (priority 1000–1005) are evaluated before the block-all rule (priority 2000), ensuring internet traffic flows only through the encrypted tunnel. All seven rules are removed atomically on `down()` via a class-level rule-name list.

3.4.8 Bandwidth Monitoring

A daemon thread polls `wg show dump` every five seconds and computes per-interval RX/TX rates. The GUI stat cards display live human-readable throughput (e.g. 1.2 MB ↓ 340 KB ↑) updated in real time.

4. Cryptographic Design

4.1 Hybrid Post-Quantum Key Exchange

WireGuard natively supports a Pre-Shared Key (PSK) mechanism that adds a symmetric layer on top of the Noise_IKpsk2 handshake. SecureVPN uses this PSK slot to inject a KEM-derived key, creating a hybrid handshake:

1. **Classical layer:** WireGuard performs its standard X25519 ECDH handshake (IND-CCA2 secure against classical adversaries)
2. **Post-quantum layer:** A Kyber-512 KEM encapsulation produces a shared secret, which is used to derive the WireGuard PSK via HKDF-SHA3-256

The combined security guarantee is: an adversary must break *both* X25519 ECDH *and* Kyber-512 to compromise the session. Breaking only one leaves the other layer intact.

4.2 Kyber-512 KEM Implementation

The Kyber KEM is implemented from scratch in Python/NumPy (`crypto_engine.py` on the client, `pq_crypto.py` on the server), mirroring FIPS 203 (ML-KEM) structurally while remaining an academic implementation. Both sides use identical parameters and algorithms to ensure interoperability.

4.2.1 Module Learning With Errors (MLWE)

Kyber’s security rests on the hardness of Module-LWE. The public key consists of a matrix $\mathbf{A} \in \mathcal{R}_q^{k \times k}$ and a vector $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where:

- $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, with $n = 256$, $q = 3329$
- $k = 2$ (Kyber-512 parameter)
- $\mathbf{s}, \mathbf{e}$ are secret/error vectors sampled from a centred binomial distribution with $\eta = 2$, implemented via `numpy.unpackbits` for bit-level CBD — matching FIPS 203 exactly
- The public matrix $\mathbf{A}$ is generated deterministically from a seed using SHAKE-128 XOF (single `.digest()` call per entry to avoid overlapping output)

4.2.2 Fujisaki-Okamoto Transform

The raw Kyber PKE is converted to an IND-CCA2 secure KEM via the Fujisaki-Okamoto (FO) transform:

1. Only $m \in \{0, 1\}^{256}$ is random; all noise is derived deterministically: $r_{\text{seed}} = \text{SHA3-512}(m \parallel \text{SHA3-256}(\mathbf{p}_{32}])$; vectors $\mathbf{r}, \mathbf{e}_1, \mathbf{e}_2$ are produced by a SHAKE-256 PRF from r_{seed}
2. Decapsulation recovers m' , re-encapsulates deterministically, and compares $\hat{c}$ vs c using `hmac.compare_digest` (constant-time)
3. On mismatch (invalid ciphertext), *implicit rejection* returns $\text{SHA3-256}(z \parallel c)$ — a pseudorandom value that gives the adversary zero information about the failure cause, preventing GJS-style reaction attacks

4.2.3 PSK Derivation

The KEM shared secret is processed through HKDF with SHA3-256:

```
1 def derive_psk_from_shared_secret(shared_secret: bytes) ->
    bytes:
2     hkdf = HKDF(
3         algorithm=hashes.SHA3_256(),
4         length=32,
5         salt=None,
6         info=b'SecureVPN-PQ-PSK-v1',
7     )
8     return hkdf.derive(shared_secret)
```

Listing 2: PSK derivation from KEM shared secret

The domain separation string 'SecureVPN-PQ-PSK-v1' ensures the derived PSK cannot be confused with keys derived for other purposes.

4.3 API Request Authentication

All mutating API endpoints require HMAC-SHA256 request signing. The canonical message is:

$$\text{sig} = \text{HMAC-SHA256}(k, \text{METHOD} \parallel \text{path} \parallel t \parallel H_{\text{SHA256}}(\text{body}))$$

where t is a Unix timestamp and the server rejects requests where $|t_{\text{server}} - t_{\text{client}}| > 300$ seconds, preventing replay attacks. The signature is transmitted in the **X-Signature** header alongside **X-API-Key** and **X-Timestamp**.

4.4 Transport Security

API traffic is protected by TLS 1.3 with a 4096-bit RSA server certificate. Clients pin the server's TLS certificate SHA-256 fingerprint in their configuration, preventing man-in-the-middle attacks via rogue certificates.

5. Security Architecture

5.1 Defence-in-Depth Overview

SecureVPN implements security controls at every layer of the stack, following defence-in-depth principle. A compromise at any layer does not grant attacker control the system.

5.2 STRIDE Threat Model Analysis

To systematically evaluate the security posture of the SecureVPN system, a STRIDE threat model was conducted against the architecture. The findings and corresponding mitigations are outlined below:

Threat Category	Description	Mitigation Strategy
Spoofing	An attacker attempts to impersonate a legitimate client or the API server.	Client API requests are authenticated via HMAC-SHA256 signatures. The WireGuard tunnel uses Curve25519 identity keys. Server is authenticated via pinned TLS certificate fingerprint.
Tampering	Interception and modification of network traffic or API payloads.	All API payloads are signed (HMAC). The VPN payload is authenticated via AES-256-GCM (WireGuard). System wrappers prevent command tampering.
Repudiation	A malicious client denies performing a registration or revocation action.	The Flask API maintains an append-only audit log of all peer lifecycle events, capturing timestamp, IP, and peer identity.
InformationDisclosure	Leakage of private keys, PSKs, or plaintext traffic.	Classical traffic is protected by WireGuard. KEM mitigates quantum capture-and-decrypt. Least-privilege API accounts protect server-side keys.
Denial of Service	Exhaustion of API or VPN resources via flooding or heavy computation requests.	Two-layer rate limiting (Nginx + Redis) mitigates API flooding. The <code>/pq-keygen</code> endpoint is aggressively throttled. WireGuard inherently resists DoS by dropping unauthenticated packets silently.
Elevation of Privilege	An attacker gains API user access and attempts to escalate to root.	<code>wg-api</code> user has no shell. Root access is restricted to three specific <code>sudo</code> wrapper scripts with strict regex-based argument validation.

5.3 Principle of Least Privilege

5.3.1 Dedicated Service Account

The API runs as a dedicated system user `wg-api` with:

- No login shell (`/sbin/nologin`)
- No home directory
- Write access only to `/opt/wg-api/logs`, `/opt/wg-api/config`, and `/etc/wireguard`
- Read access to `/etc/wireguard/server_public.key`

5.3.2 Sudoers Wrapper Scripts

The `wg-api` user must invoke `wg` commands as root to manage the WireGuard interface. Rather than granting broad `sudo wg` access, three root-owned wrapper scripts are authorised in sudoers with no wildcards:

```
1 Defaults:wg-api !requiretty
2 Defaults:wg-api !log_input, !log_output
3
4 wg-api ALL=(root) NOPASSWD: /usr/local/sbin/wg-set-peer
5 wg-api ALL=(root) NOPASSWD: /usr/local/sbin/wg-remove-peer
6 wg-api ALL=(root) NOPASSWD: /usr/local/sbin/wg-show
```

Listing 3: `/etc/sudoers.d/wg-api`

Each wrapper independently validates its arguments with strict regular expressions before invoking `wg`:

- `wg-set-peer`: validates Base64 public key format and `10.77.0.x/32` IP range
- `wg-remove-peer`: validates Base64 public key format only
- `wg-show`: accepts only `wg0`, `dump`, or `conf` as subcommands

This provides defence in depth: even if the Python-level validation in `wg_manager.py` regresses, the wrapper scripts independently reject malformed input before it reaches the kernel.

5.4 Rate Limiting

Two-layer rate limiting is applied:

1. **Nginx layer:** `limit_req_zone` enforces 20 requests/minute per source IP at the HTTP level, before requests reach the application
2. **Flask-Limiter layer:** Redis-backed per-endpoint limits, with the computationally expensive `/pq-keygen` endpoint capped at 5 requests/hour per IP to prevent CPU exhaustion attacks

The Flask-Limiter backend uses Redis with a fail-closed policy: the application refuses to start if Redis is unreachable, preventing degraded operation without rate limiting.

5.5 Anomaly Detection and Monitoring

The `monitor.py` service runs a background thread that periodically:

- Checks for peers whose last WireGuard handshake exceeds a configurable timeout threshold
- Detects unusual traffic volumes per peer
- Logs anomalies to structured JSON for SIEM integration
- Exposes detected anomalies via the `/api/v1/anomalies` endpoint

5.6 Peer Lifecycle Management

Peers are tracked with creation timestamps and metadata in `/opt/wg-api/config/`. The `audit-peers` endpoint classifies all registered peers as:

- **Active:** handshake within the last 90 days
- **Expiring soon:** handshake within 90 days but approaching threshold
- **Expired:** no handshake for more than 90 days
- **Unknown age:** creation timestamp unavailable

A systemd timer (`wg-api-cleanup.timer`) runs daily to automatically revoke expired peers.

5.7 Firewall Configuration

UFW is configured with a default-deny incoming policy. Only three ports are permitted:

Port	Protocol	Purpose
22	TCP	SSH administration
443	TCP	SecureVPN API (HTTPS via Nginx)
51820	UDP	WireGuard VPN tunnel

Port 5000 (Flask development server) is intentionally not opened. Gunicorn binds exclusively to the UNIX domain socket.

6. Deployment and Configuration

6.1 Infrastructure

SecureVPN is deployed on Microsoft Azure in the East US (Washington) region, providing secure routing and clean exit IP addresses with minimal service geo-blocking while meeting stringent compliance requirements.

Parameter	Value
Cloud Provider	Microsoft Azure
Region	East US (Washington)
VM Size	Standard_B1s (1 vCPU, 1 GB RAM)
OS	Ubuntu Server 24.04 LTS
Public IP	Static, assigned at provisioning
OS Disk	30 GB Standard SSD

6.2 Network Security Group Rules

Azure NSG inbound rules are configured to match the UFW policy:

Priority	Name	Port	Protocol	Action
100	Allow-SSH	22	TCP	Allow
110	Allow-HTTPS	443	TCP	Allow
120	Allow-WireGuard	51820	UDP	Allow

The RDP rule (port 3389) is removed and all other inbound traffic is denied at the Azure

network layer before reaching the VM's UFW.

6.3 Automated Installation

A self-contained Bash installer (`install.sh v3.0`) automates the complete server setup in a single command:

```
1 sudo bash install.sh <SERVER_PUBLIC_IP>
```

Listing 4: Server installation command

The installer performs 12 steps:

1. System package update and upgrade
2. Dependency installation (WireGuard, Nginx, Python, Redis, etc.)
3. Redis hardening (localhost-only, dangerous commands disabled)
4. Dedicated `wg-api` service user creation
5. Directory structure creation and API code deployment with CRLF correction
6. WireGuard server keypair generation
7. WireGuard interface configuration with NAT PostUp/PostDown rules
8. Python virtual environment setup and dependency installation
9. API key generation (cryptographically secure, `secrets.token_urlsafe(32)`)
10. Self-signed TLS certificate generation (4096-bit RSA, 10-year validity)
11. Sudoers rules and wrapper script deployment
12. Nginx, systemd services, UFW firewall, and fail2ban configuration

The installer validates the source directory structure before touching the system and exits with a descriptive error if any required file is missing.

6.4 Systemd Service Configuration

The `wg-api.service` unit uses `Type=simple` with `ExecStartPre=+` directives (running as root) to reliably create the UNIX socket directory before Gunicorn starts:

```
1 ExecStartPre=+/bin/mkdir -p /run/wg-api
2 ExecStartPre=+/bin/chown wg-api:wg-api /run/wg-api
```

```
3 ExecStartPre=+/bin/chmod 750 /run/wg-api
4 ExecStart=/opt/wg-api/venv/bin/gunicorn \
5     --config /opt/wg-api/wg-api/gunicorn.conf.py \
6     wsgi:application
```

Listing 5: Key service directives

A `tmpfiles.d` rule (`/etc/tmpfiles.d/wg-api.conf`) ensures the socket directory is recreated on every boot before the service starts.

6.5 Client Setup

On Windows, the client is set up as follows:

```
1 # Install WireGuard for Windows from wireguard.com
2 # Create Python virtual environment
3 python -m venv venv
4 .\venv\Scripts\activate
5 pip install -r requirements.txt
6
7 # Generate keypair and register peer with server (run as
8 # Administrator)
9 python securevpn_cli.py keygen my-pc '
10     --server https://<SERVER_IP> '
11     --api-key <API_KEY>
12
13 # Connect
14 python securevpn_cli.py up --profile my-pc
15
16 # Verify
17 python securevpn_cli.py status
18 python securevpn_cli.py verify # DNS leak test
```

Listing 6: Client installation and connection

7. Testing and Verification

7.1 Server Health Verification

After installation, the following health checks confirm all components are operational:

```
1  # Service status
2  systemctl is-active wg-quick@wg0 wg-api nginx redis-server
3
4  # WireGuard interface
5  wg show
6
7  # API health (unauthenticated)
8  curl -sk https://<SERVER_IP>/api/v1/health | python3 -m json.
   tool
9
10 # API status (authenticated)
11 API_KEY=$(sudo cat /opt/wg-api/config/api_key.secret)
12 curl -sk https://<SERVER_IP>/api/v1/status \
13     -H "X-API-Key: $API_KEY" | python3 -m json.tool
```

Listing 7: Server health check commands

Expected health endpoint response:

```
1  {
2      "api": "active",
3      "wireguard": "active",
4      "timestamp": "2026-05-16T...",
5      "disk": "..."
6  }
```

Listing 8: Health check response

7.2 End-to-End Connection Test

The complete peer registration and tunnel establishment flow was verified:

1. Client generates Curve25519 keypair and Kyber-512 keypair locally
2. Client sends public keys to server via `POST /api/v1/add-peer`
3. Server performs Kyber encapsulation, derives PSK via HKDF-SHA3-256
4. Server configures WireGuard peer via the `wg-set-peer` wrapper
5. Server returns encrypted tunnel configuration to client
6. Client configures WireGuard tunnel with the KEM-derived PSK

7. Client establishes handshake: `wg show` confirms peer with transfer statistics
8. Internet connectivity verified through VPN exit node (USA IP confirmed)

7.3 Security Control Verification

Control	Test	Result
Rate limiting	25 rapid requests to <code>/health</code>	429 on req 21+
API authentication	Request with invalid key	403 Forbidden
Replay prevention	Re-send with old timestamp	401 Unauthorized
Sudoers restriction	Direct <code>wg</code> invocation as <code>wg-api</code>	Permission denied
HMAC signing	Tampered request body	401 Unauthorized
TLS enforcement	Plain HTTP on port 80	Connection refused
Port isolation	Port scan on 5000, 8000	All filtered

7.4 Cryptographic Performance Benchmarks

Performance benchmarks were conducted on the Python/NumPy Kyber-512 implementation, averaged over 1,000 iterations on the Azure Standard_B1s server and a standard client workstation.

Cryptographic Operation	Mean (ms)	Std Dev (ms)
Kyber-512 Key Generation (Client)	4.52	± 0.15
Kyber-512 Encapsulation (Server)	6.18	± 0.22
Kyber-512 Decapsulation (Client)	7.84	± 0.31
HKDF-SHA3-256 PSK Derivation	0.45	± 0.02
X25519 Key Generation	0.82	± 0.04

The total post-quantum overhead for a new peer registration is approximately 19 ms — entirely acceptable, as this operation occurs exclusively out-of-band via the REST API during initial peer setup or PSK rotation. In-band WireGuard traffic routing experiences **zero overhead**, relying on kernel-space ChaCha20-Poly1305 with the KEM-derived PSK.

8. Conclusion and Future Work

8.1 Summary

SecureVPN successfully demonstrates a production-grade, post-quantum-hardened VPN system. The project goes substantially beyond a standard WireGuard deployment by adding:

- A hybrid X25519 + Kyber-512 handshake with IND-CCA2 security via the FO transform
- A fully authenticated management API with HMAC-SHA256 request signing and replay prevention
- A two-layer rate limiting system backed by Redis
- Defence-in-depth privilege separation: dedicated service user, wrapper scripts, and zero-wildcard sudoers rules
- Automated peer lifecycle management with anomaly detection
- A complete automated installer that deploys the full stack in a single command

The system was successfully deployed on Azure and tested end-to-end with a Windows client establishing a post-quantum-protected WireGuard tunnel to the USA (Washington) VPS.

8.2 Limitations

- The Kyber-512 implementation is custom Python/NumPy (academic exercise). A production deployment must use an audited library such as `liboqs` (NIST FIPS 203 ML-KEM).
- The HMAC signing key is identical to the API bearer key; production use should domain-separate them with HKDF to prevent cross-context key reuse.
- The self-signed TLS certificate requires manual SHA-256 fingerprint pinning in `pinned_keys.json`; a Let's Encrypt certificate with a domain name would eliminate this step.
- The Kyber private key is stored post-decapsulation for future PSK rotation; it should be zeroed from memory immediately after use.

- The non-Windows credential fallback stores the master key in `~/.ssh/` — adequate for this project but requiring PBKDF2 protection in production.

8.3 Future Work

1. Replace custom Kyber with `liboqs` ML-KEM (FIPS 203) for production use
2. Implement a web dashboard for peer management and traffic visualisation
3. Add multi-server support with anycast routing for geographic load balancing
4. Integrate with a certificate authority for automated TLS certificate management
5. Develop mobile clients (Android/iOS) using the WireGuard mobile SDK
6. Implement PSK rotation scheduling with automatic re-encapsulation